# Audion Address Processor

Русский · User Guide · OKTMO · Reference · Install

**Contents**


- Why It Exists
- How It Is Solved
- What It Can Do
- Principles
- Documentation
- Technical Reference
  - Sources
  - Settings
  - Diagnostics


Working with Russian address data: matching tables by address, assembling clean addresses from disparate sources, a reference directory, territory codes, sorting, and decomposition into parts.

## Why It Exists

An address is the worst thing to reconcile tables by — and simultaneously the only thing present in most municipal data.

The same building in two exports looks like this: `г. Сургут, ул. Ленина, д. 12` and `Сургут, Ленина 12`. Or `пр-т Мира, 5к1` and `проспект Мира, дом 5, корпус 1`. Or the address sits in one column in one table and is spread across four in another.

Matching on exact equality yields ten per cent of hits. Matching by eye costs an hour per hundred rows and produces mistakes nobody notices.

## How It Is Solved

**Five matching stages with refining passes, from strict to loose.** A row looks for its pair first by exact equality, then by parsed components, then by a normalised fingerprint, and only then approximately.

The order is not arbitrary: **a strict match never loses to a loose one**. Once a building is found exactly, fuzzy matching is not considered — otherwise a similar string from the neighbouring district would outrank the correct one. A matched candidate is removed from the pool immediately, so the same building can never be handed out twice.

The full breakdown of the passes is in the Reference.

## What It Can Do

**Align tables** against a reference address column — carrying adjacent columns along with the addresses so the data does not drift apart.

**Assemble clean addresses** from almost anything: spreadsheets, Word and ODT documents, plain text, CSV, markup, and PDF with a text layer.

**Maintain a reference directory** with slots: postal index, municipality, settlement, street, building, territory code, and technical keys.

**Sort and decompose** the directory by a chosen hierarchy of slots — from region down to building, or in any other order.

**Move columns between workbooks** by building address — not by copying rows, but through Excel, preserving formulas, formatting, sheets, and the row order of the primary workbook.

**Lean on the Rosstat OKTMO registry.** Region and municipality are chosen from the official registry, settlement names are expanded into their grammatical forms, and the search narrows down to the intended territory. The registry is downloaded by a button in the window — see OKTMO.

**Read damaged headers.** A two-level header is assembled whole: a row that carries no data values and sits under a horizontally merged parent header counts as a sub-header, whatever words it uses. A column headed "Проектная мощность, мест" over "по корпусам" becomes "Проектная мощность, мест по корпусам" instead of an unnamed `column_5`, and the sub-header row itself no longer slips into the data.

## Principles

**Sources are never modified.** Results always go into a separate folder. What came in stays as it was.

**Columns are given as in Excel.** `A`, `B`, `AC` — the same letters visible in the spreadsheet header, not ordinal numbers.

**Settings need not be edited by hand.** Everything lives in a configuration file, but columns can be set safely from the window.

**Every run leaves a report.** A JSON file appears in `report/` next to the result: what was found, by which pass, and what was left without a pair.

**The window and the command line do the same.** Any operation also runs without the window — with `address_cli.cmd`, with the same field defaults; see the Reference.

**The output spreadsheet is readable.** Column widths are fitted to the content, long text wraps, row heights grow. A small thing, but without it every export has to be tidied by hand.

## Documentation

| Document | About |
|---|---|
| User Guide | The window, workspace folders, every command step by step |
| OKTMO and project keys | Rosstat registry, download, update, cache, keys, pins |
| Technical Reference | Slots, matching passes, settings, reports |
| Install and launch | Portable environment, building the window, maintenance |
| Project file map | Where everything lives in the sources |

---

## Technical Reference

### Sources

`.xlsx`, `.docx`, `.odt`, `.txt`, `.csv`, `.md`, and `.pdf` with a text layer. Legacy `.doc` and `.xls` are converted by a separate command.

Text is read from PDF by the PyMuPDF library, which is distributed under the GNU AGPL 3.0. The program's own code stays under Apache 2.0, but the build as a whole — the program together with PyMuPDF — is offered under the terms of the GNU AGPL 3.0; see the `NOTICE` file.

## Settings

`config/project.yaml`. The window offers safe fields for the columns — editing the file is not required.

## Diagnostics

A separate breakdown of how the parser worked, and a safe removal of entirely empty rows — with no risk of catching rows that hold data in a single column.